

ECE 622: Modeling & Verification of Embedded Systems

Spring 2026: Lab 1: Reachability Analysis Report

Prof. Daniel Holcomb

Submitted by:	Shu-Lin Shih and Jaideep Khare
Student ID:	35199856 and 35193853
Lab No.:	1
Submission Date:	March 04, 2026

Overview

This lab explores explicit and symbolic reachability analysis for sequential systems. All experiments were performed on Ubuntu 16.04 using MiniSAT and PicoSAT.

Part A uses graph traversal and simulation-based reachability, while Part B implements SAT-based symbolic reachability through CNF encoding and transition relation unrolling. Part C extends the symbolic approach to counting reachable states.

1 Part A: Reachability by Graph Traversal

A.1: State Transition Graphs

For each design, we constructed the state transition graph by enumerating all states and evaluating the next-state for every possible input combination. The transitions were determined either directly from the gate-level schematic (ex1) and applying test vectors using simulation on the hdl file (stoplight1).

ex1 Design

The design `ex1.v` contains two state bits (S_1, S_0) and one input A . Therefore the system has $2^2 = 4$ possible states and $2 \times 4 = 8$ possible (A, S_1, S_0) combinations.

Using the gate-level schematic of `ex1.v`, we evaluated the next-state logic (NS_1, NS_0) for each combination. The resulting transition truth table is shown in Table 1.

A	S1	S0	NS1	NS0
0	0	0	0	1
1	0	0	0	1
0	0	1	0	1
1	0	1	0	1
0	1	0	0	1
1	1	0	0	1
0	1	1	0	0
1	1	1	1	0

Table 1: State transition truth table for `ex1.v`

From Table 1, rows that lead to identical state transitions were grouped together and represented as labeled edges in the state transition graph. The resulting FSM is shown in Fig. 1. The initial state is 00.

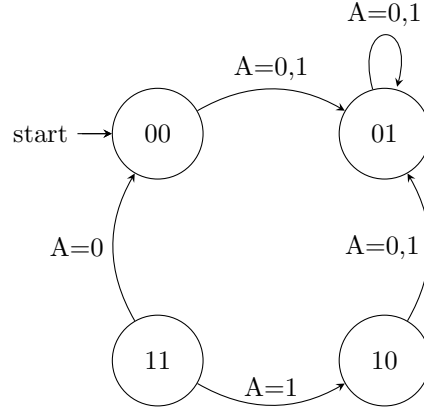


Figure 1: State transition graph for `ex1.v`

stoplight1 Design

The design `stoplight1.v` contains four state bits, resulting in $2^4 = 16$ possible states. For each state $S \in \{0000, 0001, \dots, 1111\}$ we evaluated the next state for both values of the pedestrian input signal (`Ped=0` and `Ped=1`) by simulating the design in AMD Vivado HDL Simulator screenshot as shown.

Using the transition relations obtained from simulation results (Fig. 2), we constructed the FSM shown in Fig. 3. Each node represents a 4-bit state and each directed edge is labeled with the corresponding input value of `Ped`.

S=0000	Ped=0 -> 0001	Ped=1 -> 0001
S=0001	Ped=0 -> 0010	Ped=1 -> 0010
S=0010	Ped=0 -> 0011	Ped=1 -> 0011
S=0011	Ped=0 -> 0100	Ped=1 -> 0110
S=0100	Ped=0 -> 0101	Ped=1 -> 0111
S=0101	Ped=0 -> 0101	Ped=1 -> 1000
S=0110	Ped=0 -> 0111	Ped=1 -> 0111
S=0111	Ped=0 -> 1000	Ped=1 -> 1000
S=1000	Ped=0 -> 1001	Ped=1 -> 1001
S=1001	Ped=0 -> 0000	Ped=1 -> 0000
S=1010	Ped=0 -> 1011	Ped=1 -> 1011
S=1011	Ped=0 -> 0000	Ped=1 -> 0010
S=1100	Ped=0 -> 1001	Ped=1 -> 1011
S=1101	Ped=0 -> 0000	Ped=1 -> 0000
S=1110	Ped=0 -> 1011	Ped=1 -> 1011
S=1111	Ped=0 -> 0000	Ped=1 -> 0000

Figure 2: Screenshot for `stoplight1.v` testbench simulation

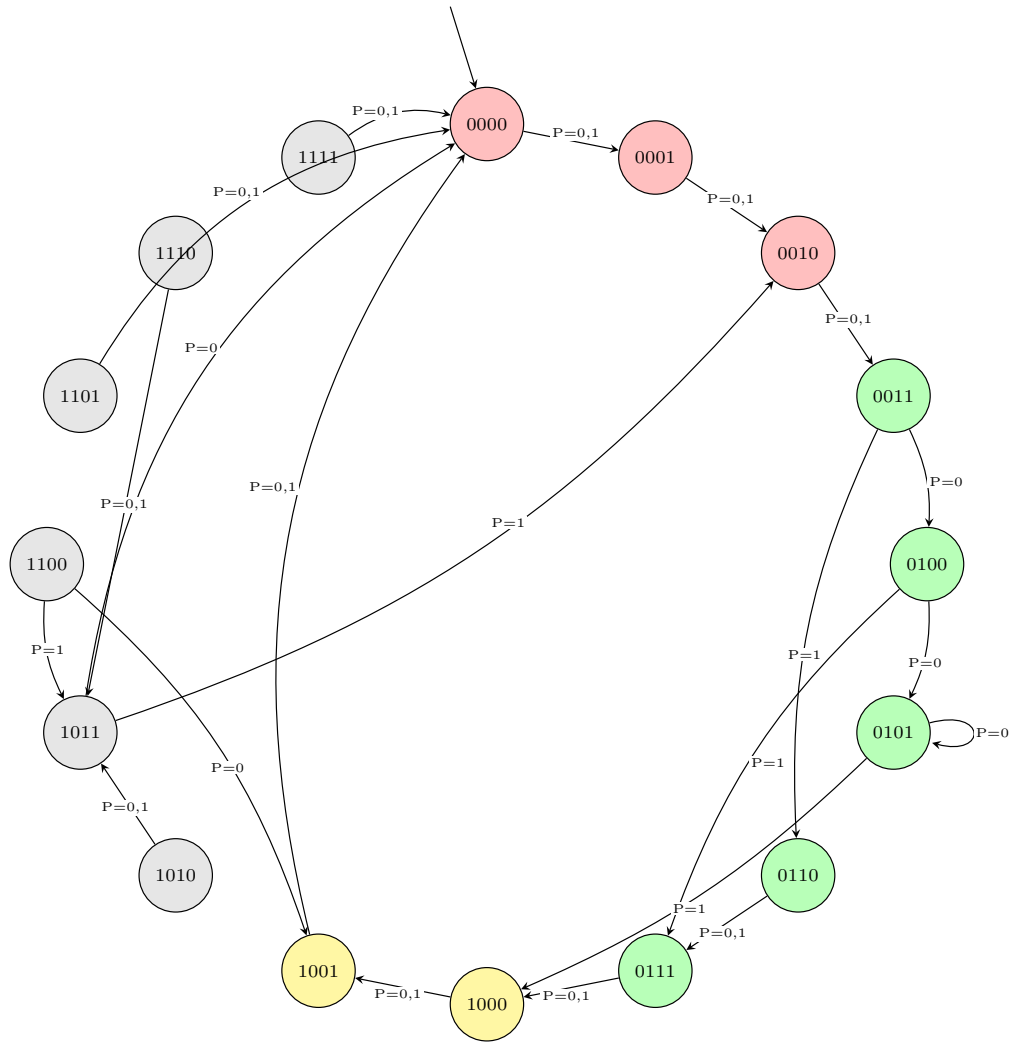


Figure 3: Complete state transition graph for **stoplight1.v**. States are color-coded for readability. Unreachability is indicated using grey-colored state circles with respect to given initial state 0000.

A.2: Target Reachability and Shortest Path

ex1: target state 11 is unreachable. From the state transition graph of **ex1.v**, the initial state is 00 and it transitions to 01 for both $A = 0$ and $A = 1$. State 01 then self-loops for both $A = 0$ and $A = 1$, so the machine becomes trapped in 01 after at most one transition. Therefore, state 11 is not reachable from the initial state 00 under any input sequence.

stoplight1: shortest path to target state 0100. From the state transition graph of **stoplight1.v**, the following shortest path (4 transitions) reaches the target state 0100 from the initial state 0000:

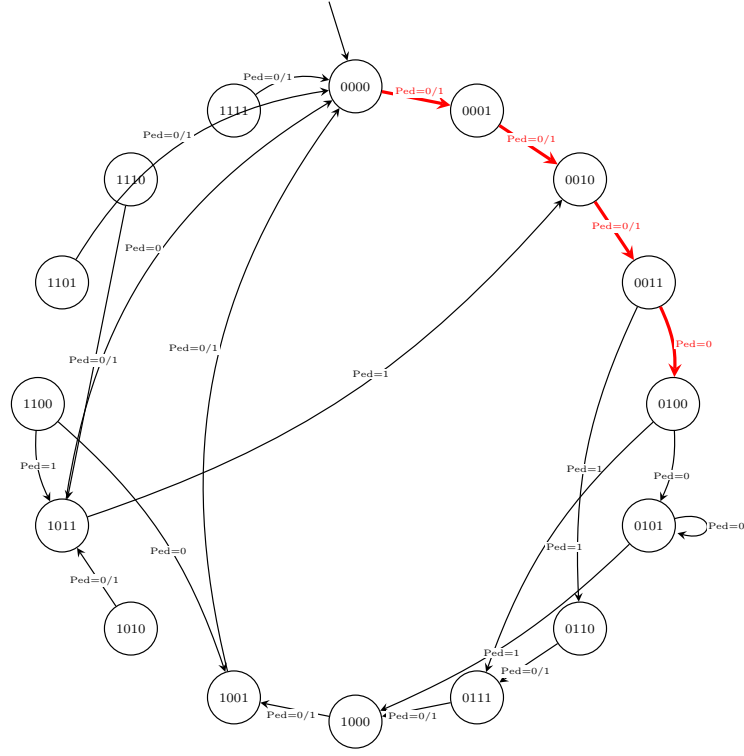


Figure 4: State transition graph for `stoplight1.v`. The shortest path from 0000 to target 0100 is highlighted in red.

A.3: States Reachable Within 20 transitions

For `ex1.v`, the reachable set saturates immediately: from initial 00, the machine reaches 01 in one step and then self-loops forever, so the number of reachable states within i transitions is constantly 2 for $i \geq 1$.

$i \in \{1, 2, \dots, 20\}$	# Reachable states within i transitions (<code>ex1</code>)
1	2 (counting initial state)
2	2
3	2
4	2
5	2
6	2
7	2
8	2
9	2
10	2
11	2
12	2
13	2
14	2
15	2
16	2
17	2
18	2
19	2
20	2

Table 2: Cumulative reachable-state counts for `ex1.v`, from the initial state {00}.

For `stoplight1.v`, the number of reachable states grows over the first few transitions and then saturates at 10 states. The increase in reachable states from 3 to 5 (when j increases from 3 to 4) occurs because different values of `Ped` introduce branching transitions that can shorten the green-light phase. This branching allows the

system to reach additional states that are not accessible under a single deterministic input sequence. Overall, the design contains a total of ten valid states: three Red states, five Green states, and two Yellow states. All of these states become reachable once the exploration depth satisfies $i \geq 7$, after which the reachable set saturates and no new states are discovered.

The growth from 4 to 6 (when j goes from 3 to 4) occurs because different values of **Ped** can introduce branch transitions (to shorten the green light), causing larger expansion of the reachable set. Total possible states include 3 Red, 5 Green and 2 Yellow states, any of which can be reached for $i \geq 7$.

$i \in \{1, 2, \dots, 20\}$	# Reachable states within i transitions (stoplight1)
1	2
2	3
3	4
4	6
5	7
6	9
7	10
8	10
9	10
10	10
11	10
12	10
13	10
14	10
15	10
16	10
17	10
18	10
19	10
20	10

Table 3: Cumulative reachable-state counts for `stoplight1.v`, from the initial state `{0000}`.

A.4: Random Execution Using Verilog Testbench

A Verilog testbench was written to explore reachability using random simulation. The testbench performs the following steps:

- Forces the initial state of the FSM to the reset state.
- Applies random input values every clock cycle.
- Checks at each cycle whether the target state has been reached.
- Terminates immediately if the target state is reached.
- Stops the experiment after 100,000 cycles if the target state is never observed.

Observed simulation outcomes:

- **ex1:** Target state 11 was **not reached** within 100,000 cycles (timeout).
- **ex2:** Target state 11111111111111 was **not reached** within 100,000 cycles (timeout).
- **stoplight1:** Target state 0100 was **reached at cycle 52**.
- **stoplight2:** Target state 01000 was **reached at cycle 1357**.

The simulation logs confirming these results are shown in the figures below.

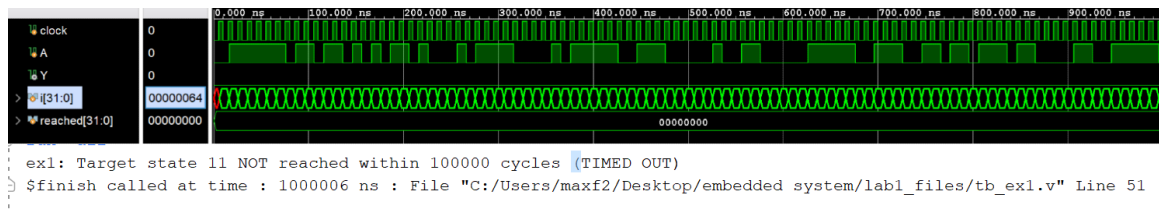


Figure 5: Waveform for `ex1`. Target state 11 was not reached within the simulation limit of 100,000 cycles.

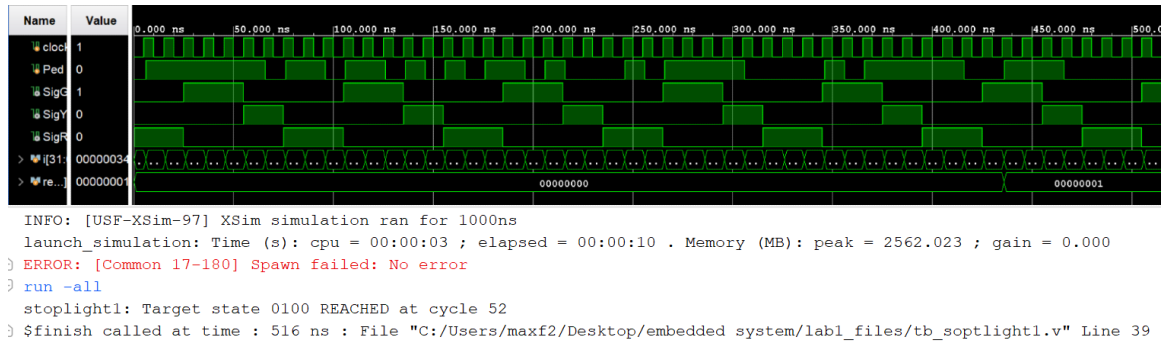


Figure 6: Waveform for `stoplight1`. Target state 0100 was reached during random simulation.

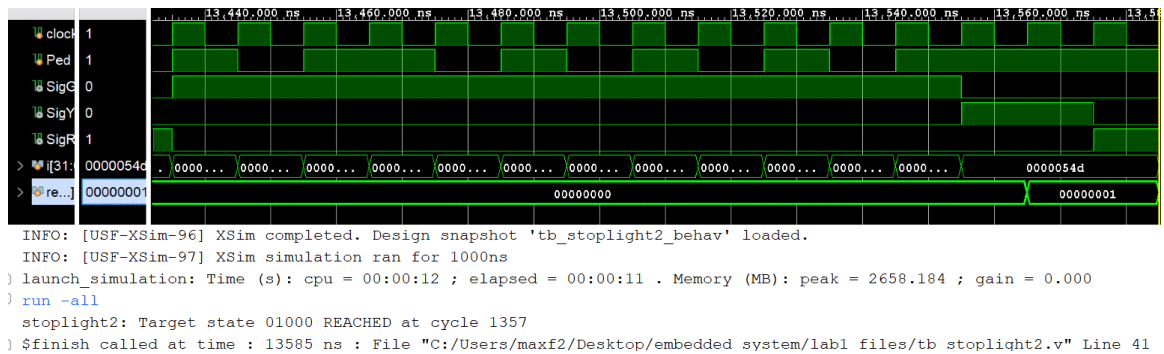


Figure 7: Waveform for `stoplight2`. Target state 01000 was reached at cycle 1357.

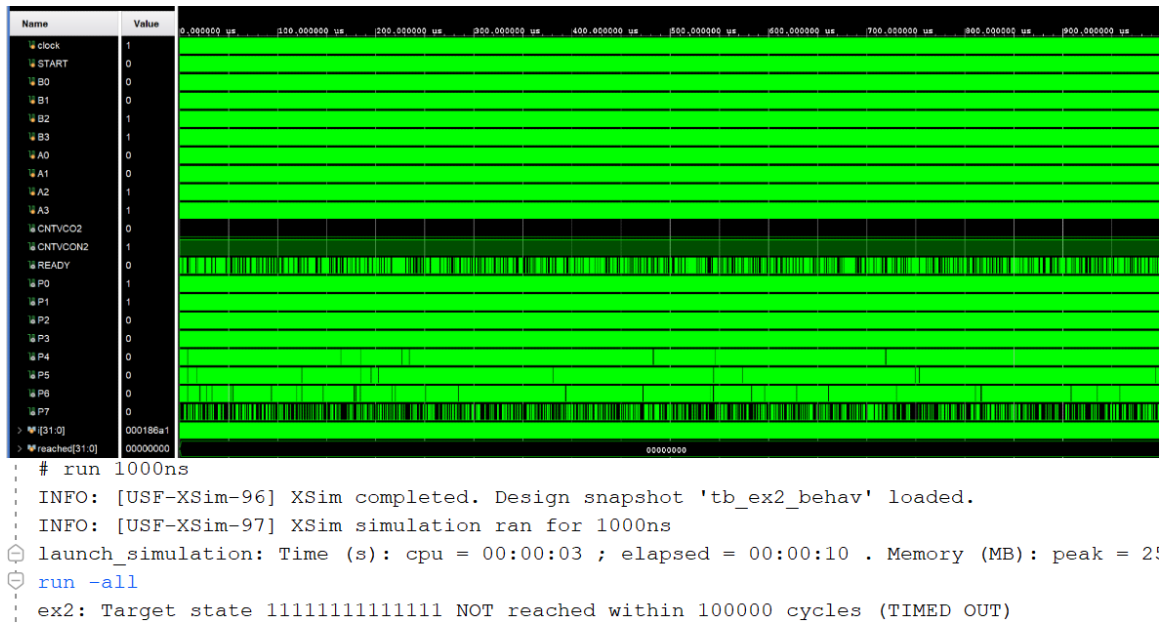


Figure 8: Waveform for ex2. Target state was not reached during the simulation interval.

We conclude that while this approach provides a practical way to observe reachability behavior in simulation, it does not guarantee exploration of all possible input sequences.

A.5: Consistency Between Graph Analysis and Simulation

For `stoplight1`, the simulation result is found to be consistent with the reachability predicted by the transition graph.

Analysis on different approaches (A.2 and A.4):

From the graph analysis in A.2, the target state 0100 is reachable from the initial state, and the shortest path requires atleast four transitions. In the random simulation experiment of A.4, the target state was reached at cycle 52. The larger number of cycles occurs because the simulation uses randomly generated inputs.

The FSM may take alternative branches in the transition graph (for example transitions through state 0110) before eventually following the sequence that leads to the target state.

A finding that would indicate an error in the earlier analysis would be if the simulation *never* reached state 0100 even after a very large number of cycles. Since the transition graph shows that this state is reachable, such a result would imply that either the graph analysis in A.2 or the simulation setup in A.4 was incorrect.

Discussion: These results illustrate an important limitation of random simulation. While random execution over many cycles can provide evidence about reachability, it cannot formally guarantee that a state is unreachable. For complex circuits, simulation may require extremely long runtimes before a rare input sequence appears.

Therefore, more systematic techniques such as symbolic execution, SAT-based bounded model checking, or formal reachability analysis are required to accurately determine circuit behavior and prove whether a state is reachable or unreachable.

2 Part B: SAT-Based Symbolic Reachability

In this part, we implemented symbolic reachability using CNF encoding and SAT solving.

Program Overview

Our C++ program performs the following steps:

1. Parse Verilog netlist.
2. Generate Boolean variables for each node.
3. Encode AND and NOT gates into CNF.

4. Unroll transition relation k times.
5. Add initial state constraints.
6. Add target state constraint at transition k.
7. Output DIMACS CNF and node mapping.

Data structures used:

- Hash map: Node $\rightarrow$ DIMACS variable index.
- Vector of clauses for CNF storage.
- Transition relation builder function.

—

B.1: ex1 Unrolled 2 Times

Checking reachability of state 11 in 2 transitions

SAT Solver Output:

<Insert SAT output here>

Result: SAT / UNSAT

Consistency with Part A: Explanation here.

Repeat for states 00, 01, 10.

—

B.2: 15-Step Reachability Analysis

Design	Reachable?	Runtime (s)
ex1	Yes/No	X.XX
ex2	Yes/No	X.XX
stoplight1	Yes/No	X.XX
stoplight2	Yes/No	X.XX

Table 4: SAT results for 15-step unrolling

Include screenshots of solver runs here.

—

B.3: stoplight1 Reachable on i-th Transition (1–32)

We automated the process using a script to invoke the solver for each value of i.

Reachable for i = list values

—

B.4: stoplight2 Comparison with Random Simulation

Symbolic results: ...

Comparison with A.4: ...

—

B.5: States Reachable on 17th Transition (stoplight2)

Approach:

- Fix transition depth to 17.
- Remove specific target constraint.
- Enumerate satisfying assignments.
- Extract state bits.

Reachable states: { }

3 Part C: Counting Reachable States (ECE 622)

C.1: Reachable States in ex5 and ex1

i	ex5 Reachable States	ex1 Reachable States
1	X	X
2	X	X
...	...	...
20	X	X

Table 5: Reachable states within i transitions

C.2: Approach Description

Our approach to counting reachable states:

- Iteratively unrolled transition relation.
- Used SAT solver to check existence.
- Added blocking clauses to enumerate unique states.
- Maintained cumulative set of reachable states.

Code submitted separately.

Use of Tools

We used the following tools in completing this lab:

- MiniSAT / PicoSAT
- Python for CNF generation
- TikZ for graph visualization
- (If applicable) ChatGPT for assistance in structuring CNF encoding and debugging.

AI tools, prompts and descriptions are included in the appendix.

Appendix

A. Prompts Used for Tool Assistance

As required by the lab instructions, we document any prompts used to assist in generating code, diagrams, or CNF encodings.

A.1 Prompt for Generating State Graph Template

Prompt:

"Generate a TikZ template for a 4-state FSM graph with labeled transitions that I can adapt for reachability analysis in Verilog."

Purpose: Used as a starting template for drawing the state transition graph for ex1.

A.2 Prompt for CNF Encoding of AND/NOT Gates

Prompt:

"Give CNF clauses for encoding a 2-input AND gate and a NOT gate in DIMACS format for SAT solving."

Purpose: Used to confirm correctness of gate-to-CNF mapping before implementing encoder.

A.3 Prompt for Unrolling Transition Relations

Prompt:

"Explain how to unroll a transition relation k times for symbolic reachability using SAT."

Purpose: Used to verify correct linking of state variables across time frames.

—

B. Verilog Testbench Code

B.1 stoplight1 Testbench

```
'timescale 1ns/1ps

module stoplight1_tb;

    reg clk;
    reg reset;
    reg input_signal;    // Placeholder input
    wire [3:0] state;    // Adjust width as needed

    integer cycle_count;

    // Instantiate DUT
    stoplight1 DUT (
        .clk(clk),
        .reset(reset),
        .input_signal(input_signal),
        .state(state)
    );

    // Clock generation
    always #5 clk = ~clk;

    initial begin
        clk = 0;
        reset = 1;
        cycle_count = 0;

        #10;
        reset = 0;

        while (cycle_count < 100000) begin
            @(posedge clk);
            input_signal = $random;
            cycle_count = cycle_count + 1;

            if (state == 4'bXXXX) begin
                $display("Target state reached at cycle %d", cycle_count);
                $finish;
            end
        end

        $display("Timed out after 100000 cycles.");
        $finish;
    end

endmodule

Replace:
```

- 4'bXXXX with actual target state.
 - Adjust signal names to match design.
-

C. SAT-Based Reachability Program (Placeholder)

C.1 Python Skeleton

```
import sys
import subprocess

def parse_verilog(filename):
    # Placeholder parser
    pass

def encode_gate_to_cnf(gate):
    # Placeholder CNF encoding
    pass

def unroll_transition(k):
    # Placeholder unrolling logic
    pass

def write_dimacs(clauses, filename):
    # Write DIMACS file
    pass

def call_sat_solver(dimacs_file):
    result = subprocess.run(
        ["picosat", dimacs_file],
        capture_output=True,
        text=True
    )
    return result.stdout

def main():
    if len(sys.argv) != 4:
        print("Usage: ~python3 lab1_src_lastname.py design.v target k")
        sys.exit(1)

    verilog_file = sys.argv[1]
    target_state = sys.argv[2]
    k = int(sys.argv[3])

    # Parse design
    parse_verilog(verilog_file)

    # Unroll transition relation
    unroll_transition(k)

    # Write CNF
    write_dimacs([], "out.dimacs")

    # Call solver
    output = call_sat_solver("out.dimacs")
    print(output)

if __name__ == "__main__":
    main()
```

—

D. Script for Automating B.3 (Loop Over $i = 1..32$)

```
#!/bin/bash

for i in {1..32}
do
    echo "Checking reachability at depth $i"
    python3 lab1_src_lastname.py stoplight1.v TARGET.STATE $i > result_${i}.txt
done
```

Replace TARGET.STATE with actual bit string.

—

E. Example DIMACS Snippet (Placeholder)

```
c Example DIMACS file
p cnf 10 15
1 -2 0
-1 3 0
2 3 -4 0
...
```

—

F. Example out.nodes Mapping File (Placeholder)

```
1: S0,0
2: S1,0
3: n3,0
4: S0,1
5: S1,1
...
```

Format: $variable_{number} : node_{name}, time_{frame}$

—

G. Additional Notes

- All CNF clauses were generated programmatically.
- Initial state constraints enforce all flip-flops = 0 at time 0.
- Target state constraint is added only at final unrolled frame.
- Node-to-variable mapping stored in out.nodes.